

Chương 1

So sánh cơ chế Indexing giữa MySQL và MongoDB

1.1 Khác biệt nền tảng trong cách sử dụng index

Điểm khác biệt quan trọng nhất giữa MySQL và MongoDB không chỉ nằm ở cú pháp tạo index, mà nằm ở mô hình dữ liệu của từng hệ quản trị. MySQL là hệ quản trị cơ sở dữ liệu quan hệ, trong đó dữ liệu được tổ chức thành bảng, dòng và cột. Các thực thể thường được chuẩn hóa thành nhiều bảng và liên kết với nhau thông qua khóa chính, khóa ngoại. Vì vậy, index trong MySQL thường được thiết kế để tối ưu các truy vấn có điều kiện lọc, truy vấn liên bảng, sắp xếp và tổng hợp dữ liệu.

Ngược lại, MongoDB lưu dữ liệu theo mô hình document BSON. Một document có thể chứa field thông thường, field lồng nhau hoặc field dạng mảng. Do đó, index trong MongoDB thường được thiết kế để tối ưu truy vấn trực tiếp trên document, embedded field hoặc array field. Sự khác biệt này làm cho MongoDB linh hoạt hơn đối với dữ liệu bán cấu trúc, nhưng cũng khiến MongoDB không tự nhiên bằng MySQL trong các truy vấn có nhiều quan hệ chặt chẽ.

Bảng 1.1: So sánh nền tảng sử dụng index giữa MySQL và MongoDB

Tiêu chí	MySQL	MongoDB
Mô hình dữ liệu	Quan hệ, bảng, dòng, cột	Document BSON, field lồng nhau, mảng
Cách tổ chức dữ liệu	Thường chuẩn hóa thành nhiều bảng	Thường gom dữ liệu liên quan vào document
Index tối ưu cho	WHERE, JOIN, ORDER BY, GROUP BY	Query trên field, embedded field, array field, aggregation pipeline
Điểm mạnh chính	Dữ liệu quan hệ, schema ổn định, truy vấn liên bảng	Dữ liệu linh hoạt, document lồng nhau, mảng, schema thay đổi

Từ bảng trên có thể thấy, MySQL phù hợp hơn khi dữ liệu có quan hệ rõ ràng và truy vấn thường xuyên liên kết nhiều bảng. MongoDB phù hợp hơn khi dữ liệu có cấu trúc linh hoạt và có thể được truy xuất theo từng document hoàn chỉnh.

1.2 So sánh index mặc định

Cả MySQL và MongoDB đều có index mặc định để hỗ trợ truy vấn theo định danh chính của dữ liệu. Tuy nhiên, vai trò của index mặc định trong hai DBMS không giống nhau.

Trong MySQL, đặc biệt với storage engine InnoDB, **PRIMARY KEY** thường đóng vai trò là clustered index. Điều này có nghĩa là dữ liệu thật của bảng được tổ chức theo giá trị khóa chính. Do đó, truy vấn theo primary key thường có hiệu năng rất tốt.

```
1 SELECT *
2 FROM products
3 WHERE product_id = 100;
```

Listing 1.1: Truy vấn theo primary key trong MySQL

Trong MongoDB, mỗi collection mặc định có index trên field `_id`. Field này đóng vai trò định danh duy nhất cho mỗi document trong collection.

```
1 db.products.find({
2   _id: ObjectId("...")
3 })
```

Listing 1.2: Truy vấn theo `_id` trong MongoDB

Điểm khác biệt là primary key trong MySQL InnoDB không chỉ dùng để định danh dòng, mà còn ảnh hưởng đến cách dữ liệu được tổ chức vật lý. Trong khi đó, `_id index` của MongoDB chủ yếu dùng để định danh và truy xuất document nhanh.

Bảng 1.2: So sánh index mặc định

Tiêu chí	MySQL InnoDB	MongoDB
Index mặc định	PRIMARY KEY	<code>_id index</code>
Đối tượng được index	Dòng trong bảng	Document trong collection
Vai trò chính	Định danh dòng và tổ chức dữ liệu vật lý	Định danh document
Ảnh hưởng đến index khác	Cao, vì secondary index liên quan đến primary key	Ít trực tiếp hơn
Nhận xét	Primary key nên ngắn, ổn định và ít thay đổi	<code>_id</code> thường tự sinh, ổn định và dễ sử dụng

1.3 So sánh secondary index và field index

Trong MySQL, các index không phải primary key thường được gọi là secondary index. Secondary index được tạo trên một hoặc nhiều cột thường xuyên xuất hiện trong truy vấn.

```
1 CREATE INDEX idx_products_sku
2 ON products(sku);
```

Listing 1.3: Secondary index trong MySQL

Khi truy vấn theo **sku**, MySQL có thể tìm trong secondary index. Nếu truy vấn cần thêm các cột không nằm trong index, InnoDB có thể phải dùng primary key được lưu trong secondary index để quay lại clustered index và lấy dòng dữ liệu đầy đủ.

```
1 SELECT *
2 FROM products
3 WHERE sku = 'SKU000123';
```

Listing 1.4: Truy vấn theo SKU trong MySQL

Trong MongoDB, index được tạo trực tiếp trên field của document.

```
1 db.products.createIndex({
2   sku: 1
3 })
```

Listing 1.5: Field index trong MongoDB

```
1 db.products.find({
2   sku: "SKU000123"
3 })
```

Listing 1.6: Truy vấn theo SKU trong MongoDB

Với các field tìm kiếm cố định như **sku**, **email**, **phone**, cả MySQL và MongoDB đều có thể tối ưu tốt bằng index. Tuy nhiên, MySQL index gắn với cấu trúc bảng và primary key, còn MongoDB index gắn trực tiếp với field trong document.

Bảng 1.3: So sánh secondary index và field index

Tiêu chí	MySQL	MongoDB
Tên gọi phổ biến	Secondary index	Field index
Tạo trên	Cột của bảng	Field của document
Cách truy xuất	Có thể phải quay lại clustered index để lấy dòng đầy đủ	Tìm document thông qua index trên field
Phù hợp với	Schema ổn định, field truy vấn cố định	Document linh hoạt, field có thể thay đổi theo document

1.4 So sánh composite index và compound index

Cả MySQL và MongoDB đều hỗ trợ index trên nhiều thuộc tính. Trong MySQL, loại này thường được gọi là composite index. Trong MongoDB, loại tương ứng được gọi là compound index.

```
1 CREATE INDEX idx_products_category_price
2 ON products(category_id, price);
```

Listing 1.7: Composite index trong MySQL

```
1 db.products.createIndex({
2   categoryId: 1,
3   price: 1
4 })
```

Listing 1.8: Compound index trong MongoDB

Hai index trên phù hợp với truy vấn lọc sản phẩm theo danh mục và khoảng giá.

```
1 SELECT *
2 FROM products
3 WHERE category_id = 3
4     AND price BETWEEN 100000 AND 500000
5 ORDER BY price;
```

Listing 1.9: Truy vấn nhiều điều kiện trong MySQL

```
1 db.products.find({
2     categoryId: 3,
3     price: { $gte: 100000, $lte: 500000 }
4 }).sort({ price: 1 })
```

Listing 1.10: Truy vấn nhiều điều kiện trong MongoDB

Điểm quan trọng là cả composite index và compound index đều phụ thuộc mạnh vào thứ tự cột hoặc field. Với index (category_id, price) hoặc { categoryId: 1, price: 1 }, DBMS có thể sử dụng tốt cho truy vấn có điều kiện trên category_id/categoryId, hoặc vừa có category vừa có price. Nếu truy vấn chỉ lọc theo price, index này thường không tối ưu vì price không phải thuộc tính đầu tiên trong index.

Bảng 1.4: So sánh composite index và compound index

Loại truy vấn	MySQL (category_id, price)	MongoDB {categoryId: 1, price: 1}
Lọc theo category	Dùng tốt	Dùng tốt
Lọc theo category và price	Dùng tốt	Dùng tốt
Lọc chỉ theo price	Không tối ưu	Không tối ưu
Lọc category và sắp xếp theo price	Có thể hỗ trợ tốt	Có thể hỗ trợ tốt

Như vậy, trong cả hai DBMS, index nhiều thuộc tính chỉ hiệu quả khi thứ tự field trong index phù hợp với query pattern thực tế.

1.5 So sánh covering index và covered query

Một trường hợp tối ưu hơn secondary index thông thường là khi toàn bộ dữ liệu cần cho truy vấn đều nằm trong index. Khi đó, DBMS có thể trả kết quả trực tiếp từ index mà không cần truy cập lại bảng hoặc collection.

Trong MySQL, trường hợp này thường được gọi là covering index.

```
1 CREATE INDEX idx_products_sku_name
2 ON products(sku, name);
3
4 SELECT sku, name
```

```

5 FROM products
6 WHERE sku = 'SKU000123';

```

Listing 1.11: Covering index trong MySQL

Truy vấn trên chỉ cần hai cột **sku** và **name**, trong khi cả hai đã nằm trong index. Vì vậy, MySQL có thể giảm việc đọc lại dữ liệu từ bảng chính.

Trong MongoDB, khái niệm tương ứng là covered query. Một truy vấn được xem là covered khi các field trong điều kiện tìm kiếm và projection đều nằm trong index. Ngoài ra, do MongoDB mặc định trả về **_id**, nếu **_id** không nằm trong index thì cần loại bỏ field này trong projection.

```

1 db.products.createIndex({
2   sku: 1,
3   name: 1
4 })
5
6 db.products.find(
7   { sku: "SKU000123" },
8   { _id: 0, sku: 1, name: 1 }
9 )

```

Listing 1.12: Covered query trong MongoDB

Bảng 1.5: So sánh covering index và covered query

Tiêu chí	MySQL	MongoDB
Tên gọi	Covering index	Covered query
Điều kiện	Các cột trong SELECT và WHERE nằm trong index	Các field trong filter và projection nằm trong index
Lưu ý đặc biệt	Tránh đọc lại bảng chính	Cần loại _id nếu _id không nằm trong index
Lợi ích	Giảm I/O, tăng tốc truy vấn đọc	Giảm số document phải đọc, tăng tốc query

Covering index và covered query đều là kỹ thuật quan trọng khi tối ưu các truy vấn đọc thường xuyên và chỉ cần một số field nhất định.

1.6 So sánh partial index

MongoDB hỗ trợ partial index trực tiếp thông qua **partialFilterExpression**. Cơ chế này cho phép chỉ đánh index trên những document thỏa một điều kiện nhất định.

```

1 db.orders.createIndex(
2   { userId: 1, orderDate: -1 },
3   { partialFilterExpression: { status: "ACTIVE" } }
4 )

```

Listing 1.13: Partial index trong MongoDB

Ví dụ trên chỉ index các đơn hàng có trạng thái **ACTIVE**. Điều này giúp giảm kích thước index, tiết kiệm bộ nhớ và giảm chi phí ghi dữ liệu, vì không phải document nào cũng cần được đưa vào index.

MySQL không hỗ trợ partial index theo kiểu filtered index trực tiếp giống MongoDB. Tuy nhiên, từ MySQL 8.0.13, MySQL hỗ trợ functional key parts, cho phép tạo index trên biểu thức. Ngoài ra, người dùng có thể dùng generated column hoặc virtual column để mô phỏng một số tình huống tương tự.

```
1 CREATE INDEX idx_active_orders_user
2 ON orders ((CASE WHEN status = 'ACTIVE' THEN user_id ELSE NULL
   END)));
```

Listing 1.14: Mô phỏng một phần partial index trong MySQL

Cách tiếp cận này không hoàn toàn tương đương partial index của MongoDB, nhưng có thể giúp tối ưu một số truy vấn có điều kiện cố định.

Bảng 1.6: So sánh partial index

Tiêu chí	MySQL	MongoDB
Hỗ trợ trực tiếp	Không hỗ trợ filtered partial index trực tiếp	Có hỗ trợ trực tiếp
Cách làm	Dùng expression index, generated column hoặc virtual column để mô phỏng	Dùng <code>partialFilterExpression</code>
Mức độ linh hoạt	Thấp hơn	Cao hơn
Lợi ích chính	Tối ưu được một số điều kiện cố định	Giảm kích thước index và chi phí ghi rõ rệt

Với các hệ thống có nhiều dữ liệu lịch sử nhưng chỉ thường xuyên truy vấn dữ liệu đang hoạt động, MongoDB có lợi thế hơn nhờ partial index.

1.7 So sánh index cho truy vấn quan hệ và dữ liệu mảng

MySQL có lợi thế rõ trong các truy vấn quan hệ vì JOIN là thao tác tự nhiên của cơ sở dữ liệu quan hệ. Index trên khóa chính, khóa ngoại và cột lọc giúp tối ưu mạnh quá trình join.

```
1 SELECT o.order_id, o.order_date, o.total_amount
2 FROM orders o
3 JOIN users u ON o.user_id = u.user_id
4 WHERE u.email = 'test@gmail.com';
```

Listing 1.15: Truy vấn JOIN trong MySQL

Các index phù hợp là:

```
1 CREATE INDEX idx_users_email
2 ON users(email);
3
```

```

4 CREATE INDEX idx_orders_user_id
5 ON orders(user_id);

```

Listing 1.16: Index hỗ trợ JOIN trong MySQL

Trong MongoDB, thao tác tương tự có thể thực hiện bằng `$lookup`, nhưng đây thường không phải hướng thiết kế chính. MongoDB thường ưu tiên nhúng dữ liệu hoặc denormalize để giảm nhu cầu join.

Ngược lại, MongoDB có lợi thế rõ khi dữ liệu chứa mảng. Ví dụ, một sản phẩm có nhiều tag:

```

1 {
2   name: "Organic Milk",
3   tags: ["food", "drink", "organic"]
4 }

```

Listing 1.17: Array field trong MongoDB

MongoDB có thể tạo multikey index trực tiếp trên field `tags`.

```

1 db.products.createIndex({
2   tags: 1
3 })

```

Listing 1.18: Multikey index trong MongoDB

Trong MySQL, để lưu nhiều tag cho một sản phẩm theo hướng chuẩn hóa, thường cần bảng phụ và truy vấn join.

```

1 CREATE TABLE product_tags (
2   product_id INT,
3   tag VARCHAR(50),
4   PRIMARY KEY(product_id, tag)
5 );

```

Listing 1.19: Lưu tag trong MySQL bằng bảng phụ

Bảng 1.7: So sánh index cho JOIN và dữ liệu mảng

Tiêu chí	MySQL	MongoDB
Truy vấn quan hệ	Mạnh, tự nhiên với JOIN	Có <code>\$lookup</code> , nhưng thường ưu tiên denormalize
Index hỗ trợ quan hệ	Index trên PK, FK, cột lọc	Index trên field lookup hoặc field denormalized
Dữ liệu mảng	Thường cần bảng phụ	Hỗ trợ trực tiếp bằng multikey index
Trường hợp phù hợp	Dữ liệu chuẩn hóa, nhiều quan hệ	Document có field lồng nhau hoặc array

Do đó, MySQL phù hợp hơn khi truy vấn cần kết hợp nhiều bảng. MongoDB phù hợp hơn khi dữ liệu có thể gom vào document hoặc chứa nhiều mảng.

1.8 So sánh specialized indexes

Ngoài các index thông thường, cả MySQL và MongoDB đều hỗ trợ một số index đặc thù như text index và geospatial index.

Với tìm kiếm văn bản, MySQL hỗ trợ **FULLTEXT index** trên các cột văn bản như tên sản phẩm hoặc mô tả.

```
1 CREATE FULLTEXT INDEX idx_products_text
2 ON products(name, description);
```

Listing 1.20: FULLTEXT index trong MySQL

MongoDB hỗ trợ text index cho truy vấn văn bản trên field dạng chuỗi.

```
1 db.products.createIndex({
2   name: "text",
3   description: "text"
4 })
```

Listing 1.21: Text index trong MongoDB

Với dữ liệu không gian, MySQL có spatial index, còn MongoDB hỗ trợ geospatial index như **2dsphere**. Nếu hệ thống cần chức năng tìm cửa hàng hoặc kho hàng gần vị trí người dùng, MongoDB thường thuận lợi khi dữ liệu vị trí được lưu trực tiếp trong document.

```
1 db.stores.createIndex({
2   location: "2dsphere"
3 })
```

Listing 1.22: 2dsphere index trong MongoDB

Bảng 1.8: So sánh specialized indexes

Loại index	MySQL	MongoDB
Text search	FULLTEXT index	Text index
Geospatial	Spatial index	2dsphere index
Điểm mạnh	Phù hợp dữ liệu quan hệ có cột văn bản/không gian rõ ràng	Tự nhiên hơn với document chứa vị trí, field linh hoạt

MongoDB không phải là DBMS duy nhất hỗ trợ dữ liệu không gian, nhưng mô hình document giúp việc lưu và truy vấn dữ liệu vị trí trở nên tự nhiên hơn trong nhiều ứng dụng location-based.

1.9 So sánh cách kiểm tra index

Trong thực nghiệm, tạo index chưa đủ để kết luận truy vấn đã được tối ưu. Cần kiểm tra execution plan để biết DBMS có thực sự sử dụng index hay không.

Với MySQL, có thể sử dụng **EXPLAIN** hoặc **EXPLAIN ANALYZE**.

```
1 EXPLAIN ANALYZE
2 SELECT *
```



```

3 FROM products
4 WHERE sku = 'SKU000123';

```

Listing 1.23: Kiểm tra index trong MySQL

Các thông tin cần quan sát gồm **key**, **rows**, **type** và **Extra**. Nếu truy vấn sử dụng index tốt, số dòng cần đọc thường giảm đáng kể.

Với MongoDB, có thể sử dụng `explain("executionStats")`.

```

1 db.products.find({
2   sku: "SKU000123"
3 }).explain("executionStats")

```

Listing 1.24: Kiểm tra index trong MongoDB

Các thông tin quan trọng gồm **winningPlan**, **stage**, **totalKeysExamined**, **totalDocsExamined** và **executionTimeMillis**. Nếu MongoDB hiển thị **COLLSCAN**, truy vấn đang quét toàn bộ collection. Nếu hiển thị **IXSCAN**, truy vấn đã sử dụng index.

Bảng 1.9: So sánh cách kiểm tra index

Nội dung	MySQL	MongoDB
Công cụ kiểm tra	EXPLAIN, EXPLAIN ANALYZE	explain("executionStats")
Dấu hiệu dùng index	Có index trong cột key	Có stage IXSCAN
Dấu hiệu không tối ưu	Quét nhiều rows	COLLSCAN
Chỉ số cần ghi nhận	Rows examined, execution time	totalDocsExamined, totalKeysExamined, executionTimeMillis

1.10 So sánh tác động đến ghi dữ liệu và bộ nhớ

Index giúp tăng tốc truy vấn đọc nhưng làm tăng chi phí ghi. Khi insert, update hoặc delete dữ liệu, DBMS không chỉ cập nhật dữ liệu chính mà còn phải cập nhật các index liên quan. Vì vậy, tạo quá nhiều index có thể làm chậm thao tác ghi và tăng dung lượng lưu trữ.

Trong MySQL, nếu bảng **products** có nhiều index như **idx_products_sku**, **idx_products_category**, **idx_products_name**, mỗi thao tác ghi đều có thể phải cập nhật nhiều cấu trúc index. MongoDB cũng tương tự: nếu collection có nhiều index như **{sku: 1}**, **{categoryId: 1, price: 1}**, **{tags: 1}**, mỗi document được thêm hoặc cập nhật đều có thể kéo theo chi phí cập nhật index.

Một điểm riêng của MySQL InnoDB là Adaptive Hash Index. Đây là cơ chế tối ưu nội bộ, trong đó InnoDB có thể tự tạo hash index trong bộ nhớ cho một số page được truy cập thường xuyên. Với workload có nhiều truy vấn điểm lặp lại, cơ chế này có thể giúp giảm chi phí tra cứu so với việc chỉ dựa vào B-Tree/B+Tree. MongoDB không có cơ chế hoàn toàn tương đương Adaptive Hash Index; hiệu năng đọc của MongoDB phụ thuộc nhiều vào WiredTiger cache và filesystem cache.

Bảng 1.10: So sánh tác động của index đến ghi dữ liệu và bộ nhớ

Tiêu chí	MySQL	MongoDB
Lợi ích chính	Tăng tốc truy vấn đọc	Tăng tốc truy vấn đọc
Chi phí ghi	Tăng khi có nhiều index	Tăng khi có nhiều index
Chi phí lưu trữ	Index chiếm thêm dung lượng	Index chiếm thêm dung lượng
Tối ưu bộ nhớ riêng	Adaptive Hash Index trong InnoDB	WiredTiger cache và filesystem cache
Cách thiết kế hợp lý	Tạo index theo query pattern	Tạo index theo query pattern

1.11 Bảng tổng hợp so sánh

Bảng 1.11: Tổng hợp so sánh indexing giữa MySQL và MongoDB

Tiêu chí	MySQL	MongoDB	Nhận xét
Mô hình dữ liệu	Quan hệ, bảng, dòng, cột	Document BSON	Quyết định cách thiết kế index
Index mặc định	Primary key, clustered index trong InnoDB	<code>_id index</code>	Cả hai tối ưu truy vấn theo định danh
Index phụ	Secondary index trên cột	Index trên field document	MySQL gắn với primary key, MongoDB gắn với field
Index nhiều thuộc tính	Composite index	Compound index	Cả hai phụ thuộc thứ tự cột hoặc field
Covering index	Covering index	Covered query	Giảm truy cập lại bảng/collection
Partial index	Không hỗ trợ trực tiếp kiểu <code>filtered index</code> ; có thể mô phỏng bằng <code>expression/generated column</code>	Hỗ trợ trực tiếp bằng <code>partialFilterExpression</code>	MongoDB linh hoạt hơn
Truy vấn join	Tối ưu tốt bằng index trên PK/FK	Có <code>\$lookup</code> , nhưng thường ưu tiên <code>denormalize</code>	MySQL có lợi thế hơn
Dữ liệu array	Thường cần bảng phụ	Hỗ trợ <code>multikey index</code>	MongoDB có lợi thế hơn
Text search	<code>FULLTEXT index</code>	Text index	Cả hai đều hỗ trợ
Geospatial	Spatial index	<code>2dsphere index</code>	MongoDB thuận lợi khi dữ liệu vị trí nằm trong document
Kiểm tra index	<code>EXPLAIN</code> , <code>EXPLAIN ANALYZE</code>	<code>explain("executionStats")</code>	<code>explain("executionStats")</code> kiểm chứng được query plan
Tác động đến ghi	Nhiều index làm chậm ghi	Nhiều index làm chậm ghi	Điểm chung

1.12 Kết luận so sánh

Qua các tiêu chí trên, có thể thấy MySQL và MongoDB đều hỗ trợ nhiều cơ chế index để cải thiện hiệu năng truy vấn. Tuy nhiên, hiệu quả của index phụ thuộc mạnh vào mô hình dữ liệu và kiểu truy vấn chính của ứng dụng.

MySQL phù hợp hơn với các hệ thống có dữ liệu quan hệ rõ ràng, schema ổn định và thường xuyên cần join giữa nhiều bảng. Index trong MySQL phát huy hiệu quả khi

được thiết kế trên primary key, foreign key, các cột lọc và các cột phục vụ báo cáo tổng hợp. Trong các hệ thống bán hàng được chuẩn hóa thành **users**, **products**, **orders**, **order_items**, MySQL là lựa chọn hợp lý nếu hệ thống cần nhiều truy vấn join và báo cáo SQL.

MongoDB phù hợp hơn với dữ liệu document, dữ liệu lồng nhau, dữ liệu dạng mảng hoặc dữ liệu có schema thay đổi linh hoạt. Các cơ chế như compound index, multikey index, covered query và partial index giúp MongoDB tối ưu tốt các truy vấn trực tiếp trên document. Với các hệ thống lưu đơn hàng dưới dạng document, sản phẩm có nhiều tag, attributes hoặc variants, MongoDB có lợi thế về mức độ tự nhiên trong thiết kế dữ liệu và truy vấn.

Vì vậy, kết luận quan trọng không phải là MySQL hay MongoDB có index tốt hơn tuyệt đối. MySQL mạnh hơn trong bối cảnh dữ liệu quan hệ và truy vấn join, trong khi MongoDB mạnh hơn trong bối cảnh dữ liệu document, dữ liệu lồng nhau, dữ liệu dạng mảng và các truy vấn linh hoạt trên field. Việc lựa chọn DBMS cần dựa trên query pattern, mô hình dữ liệu và yêu cầu mở rộng của ứng dụng.

Tài liệu tham khảo

- [1] Chapter 3 – Indexing Structures for Files, Course Material.
- [2] Oracle Corporation, *MySQL Reference Manual: Optimization and Indexes*. Available at: <https://dev.mysql.com/doc/refman/8.4/en/optimization-indexes.html>
- [3] Oracle Corporation, *MySQL Reference Manual: InnoDB Indexes*. Available at: <https://dev.mysql.com/doc/refman/8.4/en/innodb-indexes.html>
- [4] MongoDB Inc., *MongoDB Manual: Indexes*. Available at: <https://www.mongodb.com/docs/manual/indexes/>
- [5] MongoDB Inc., *MongoDB Manual: Partial Indexes*. Available at: <https://www.mongodb.com/docs/manual/core/index-partial/>